Lesson 2 - Projections, lighting and transformations

António Alberto - 114622

Information Visualization, 2025 (MSc Informatic Engineering, University of Aveiro)

I. Multiple actors

Q: “Change the opacity of the two cones to 0.5, what do you observe?”

A: Initially we have the two cones (Fig. 1):

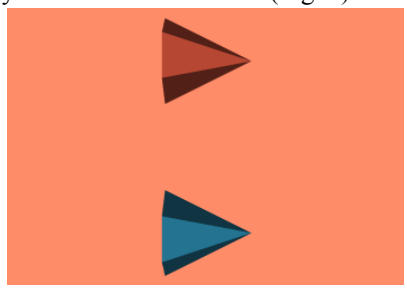

Fig. 1 - Cones initially.

Then, after modifying the opacity of the two cones, they become more transparent (Fig. 2).

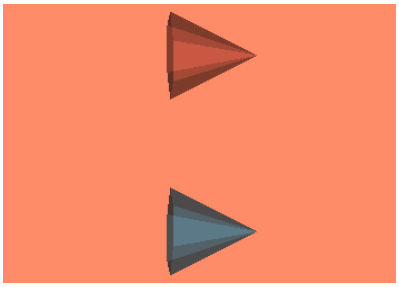

Fig. 2 - Cones with SetOpacity(0.5).

Additionally, we can see one cone through the other, since they are transparent (Fig. 3).

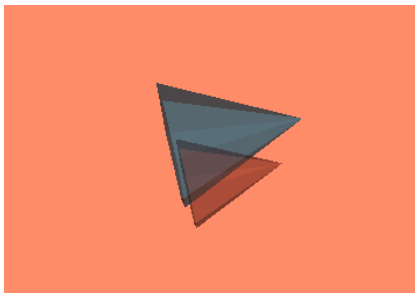

Fig. 3 - Seeing one cone through the other.

II. Multiple renderers

Q: “Define another renderer in the same window. Modify the window size to 600 by 300 pixels , with half the window for each renderer. Modify the initial position of the camera in the second renderer so that it has a 90 degree rotation in azimuth, and then make the cone rotate in both windows by adapting the code from the first example of lesson 1. Modify the background color in each Viewport.”

A: The visualization window becomes the one below (Fig. 4), and the code I added was:

To define the two renderers, each one with half the window, and change their background colors:

```
# left half
ren1 = vtkRenderer()
ren1.AddActor(coneActor)
ren1.SetBackground(0.1, 0.2, 0.4)
ren1.SetViewport(0.0, 0.0, 0.5, 1.0)

# right half
ren2 = vtkRenderer()
ren2.AddActor(coneActor)
ren2.SetBackground(0.2, 0.3, 0.4)
ren2.SetViewport(0.5, 0.0, 1.0, 1.0)
```

Then, to modify the window size and rotate the camera on the second renderer:

```
ren1.ResetCamera()
ren2.ResetCamera()
ren2.GetActiveCamera().Azimuth(90)

renWin = vtkRenderWindow()
renWin.SetSize(600, 300)
```

To make the cone rotate, like in lesson 1, by rotating the cameras around the cone:

```
for i in range(0, 360):
    renWin.Render()
    ren1.GetActiveCamera().Azimuth(1)
    ren2.GetActiveCamera().Azimuth(1)
```

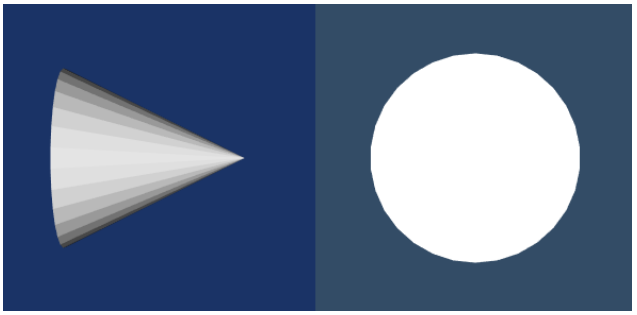

Fig. 4 - The same cone in the two renderers.

III. Shading options

Q: "Modify the code to display two spheres instead of two cones. Change the code in such a way that the shading of the second sphere becomes flat. What do you observe? Try the other options (Gouraud, Phong)."

A: I decided to modify the code to show 4 spheres. The first one is the default shading option, which is the same as the last one, because the default shading option is Gouraud. The one on the top-right has flat shading. The one on the left-bottom has Phong shading. We can observe that the different shading options change the way the object is represented (Fig. 5). The most important part of the code is the following, which is where the shading options are defined:

```
sphereActor1 = vtkActor()
sphereActor1.SetMapper(sphereMapper)

sphereActor2 = vtkActor()
sphereActor2.SetMapper(mapper_flat)
sphereActor2.GetProperty().SetInterpolation
ToFlat()

sphereActor3 = vtkActor()
sphereActor3.SetMapper(sphereMapper)
sphereActor3.GetProperty().SetInterpolation
ToPhong()

sphereActor4 = vtkActor()
sphereActor4.SetMapper(sphereMapper)
sphereActor4.GetProperty().SetInterpolation
ToGouraud()
```

Note: I had to change the mapper for the flat shading one to render correctly whenever I rotated the sphere or changed the window size manually (e.g. full-screened the VTK window):

```
normalsFlat = vtkPolyDataNormals()
normalsFlat.SetInputConnection(sphereSource
.GetOutputPort())
normalsFlat.ComputePointNormalsOff()
normalsFlat.Update()

mapperFlat = vtkPolyDataMapper()
mapperFlat.SetInputConnection(normalsFlat.G
etOutputPort())
```

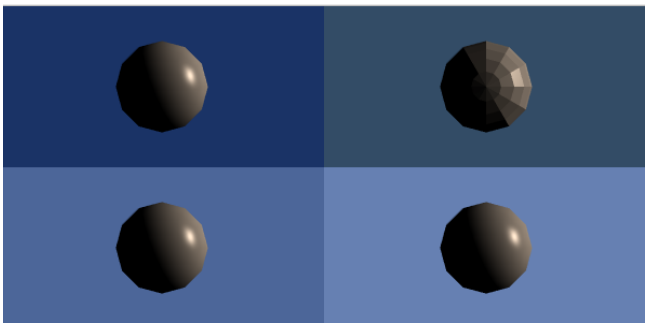

Fig. 5 - The same sphere, rendered in four different renderers. The first and last one have Gouraud shading. The top-right one has flat shading. The bottom-left one has Phone shading.

IV. Textures

Q: "Make a VTK program that allows to visualize a plane. Analyze the vtkTexture class and use the VTKXXXReader class to read an image and map it to the created plane. Try changing the size of the Plane (you can use the setOrigin, SetPoint1 and SetPoint2 methods), what happens to the texture?"

A: I created the plane with the image initially with the following code:

```
# Create a plane
planeSource = vtkPlaneSource()
planeSource.Update()

# Read the texture image
jpgReader = vtkJPEGReader()
jpgReader.SetFileName("./images/lena.JPG")
jpgReader.Update()

# Create texture object
texture = vtkTexture()
texture.SetInputConnection(jpgReader.GetOut
putPort())

# Create mapper and actor
planeMapper = vtkPolyDataMapper()
planeMapper.SetInputConnection(planeSource.
GetOutputPort())

planeActor = vtkActor()
planeActor.SetMapper(planeMapper)
planeActor.SetTexture(texture)
```

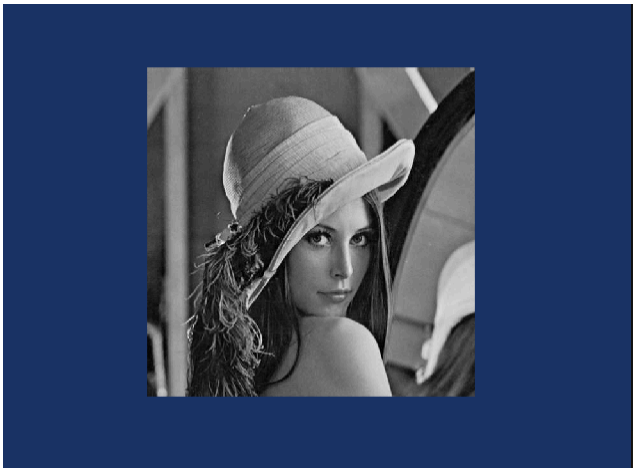

Fig.6 - The plane.

It is possible to rotate the plane (Fig. 7):

Fig. 7 - The plane with the image, rotated.

Q: “Try changing the size of the Plane, what happens to the texture?”
A: I made the plane have double the width, and the texture got deformed; it adapted to the plane’s dimension (Fig. 8).

```
planeSource = vtkPlaneSource()
planeSource.SetOrigin(0, 0, 0)
planeSource.SetPoint1(2, 0, 0)
planeSource.SetPoint2(0, 1, 0)
planeSource.Update()
```

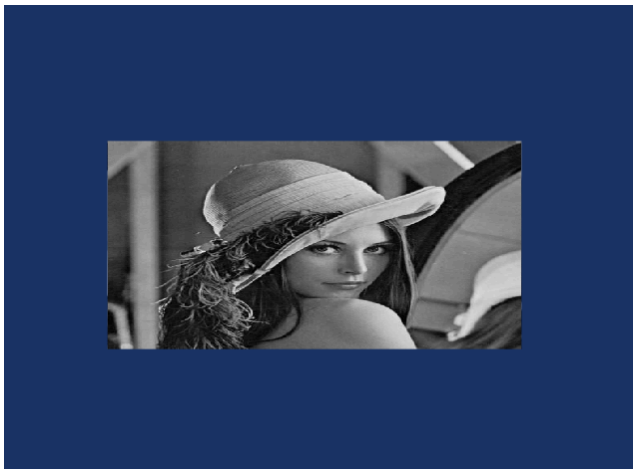

Fig. 7 - The plane with the new dimensions.

V. Transformation

Q: “Create six planes and use six different texture images for each one, then use the vtkTransform class to move the planes in such a way that they form a cube. To ease the process, create a function that, given the rotation, translation and name of the image for texture, returns the desired actor.

A: First, I created a function to make the planes:

```
 def createTexturedPlane(texture_file,
translation=(0,0,0), rotation=(0,0,0)):
    # Create plane
    plane = vtkPlaneSource()
    plane.Update()
    # Apply texture
   reader = vtkJPEGReader()
   reader.SetFileName(texture_file)
   reader.Update()

   texture = vtkTexture()

texture.SetInputConnection(reader.GetOutputPort
())

   # Apply transformation
   transform = vtkTransform()
   transform.Translate(*translation)
   transform.RotateX(rotation[0])
   transform.RotateY(rotation[1])
   transform.RotateZ(rotation[2])

   transformFilter =
vtkTransformPolyDataFilter()
   transformFilter.SetTransform(transform)

transformFilter.SetInputConnection(plane.GetOut
putPort())
   transformFilter.Update()

   # Mapper and actor
   mapper = vtkPolyDataMapper()

mapper.SetInputConnection(transformFilter.GetOu
tputPort())

   actor = vtkActor()
   actor.SetMapper(mapper)
   actor.SetTexture(texture)

   return actor
```

And then, to make the cube:

```
      transforms = [
          ((0, 0, 0.5), (0, 0, 0)),      #
   front
          ((0, 0, -0.5), (0, 180, 0)),   #
   back
          ((-0.5, 0, 0), (0, -90, 0)),   #
   left
          ((0.5, 0, 0), (0, 90, 0)),     #
   right
          ((0, 0.5, 0), (-90, 0, 0)),    # top
          ((0, -0.5, 0), (90, 0, 0)),   #
   bottom
      ]

      # Create and add actors
      for i in range(6):
          actor =
   createTexturedPlane(images[i],
   transforms[i][0], transforms[i][1])
          ren.AddActor(actor)
```

The final output is the desired cube (Fig. 8):

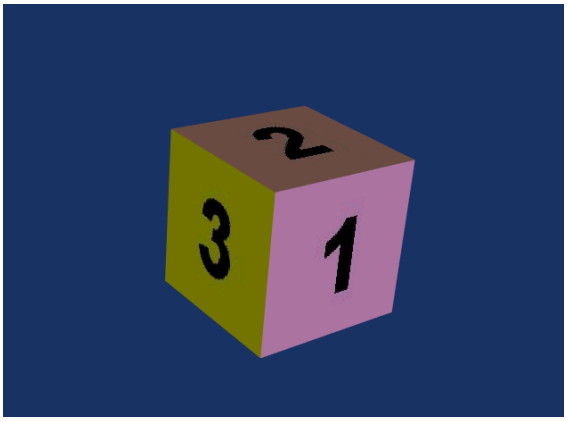

Fig. 8 - The cube with different textures in each face.

VI. Use of callback for interaction

Q: “Add the function in the cone with interaction example. What do you observe when running the program?”

A: All the events are outputted to the terminal (Fig. 9). To change the type of event observed, it is in the AddObserver function:

```
ren.AddObserver(vtkCommand.EndEvent,mo1)
```

```
vtkOpenGLRenderer Event Id: ComputeVisiblePropBoundsEvent
Camera Position: 0.843774, 3.050964, -0.197249
vtkOpenGLRenderer Event Id: ResetCameraClippingRangeEvent
Camera Position: 0.843774, 3.050964, -0.197249
vtkOpenGLRenderer Event Id: StartEvent
Camera Position: 0.843774, 3.050964, -0.197249
vtkOpenGLRenderer Event Id: EndEvent
Camera Position: 0.843774, 3.050964, -0.197249
vtkOpenGLRenderer Event Id: ModifiedEvent
Camera Position: 0.843774, 3.050964, -0.197249
```

Fig. 9 - The output of interacting with the cube window.